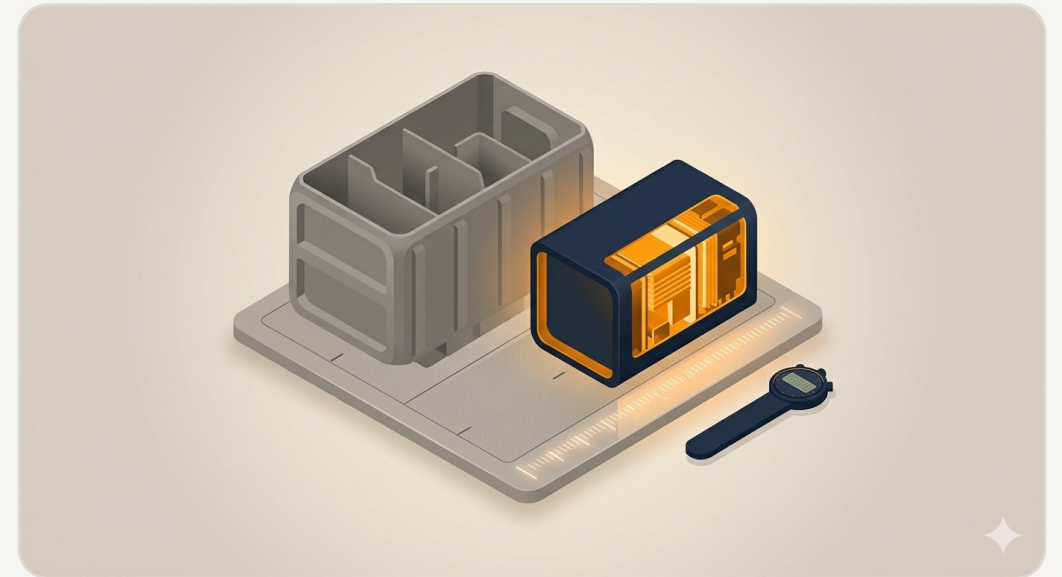


---

# Things Docker can do that you haven't tried

BuildKit features hiding behind  
commands you use every day

DEVOPS INTERNSHIP · WEEK 6



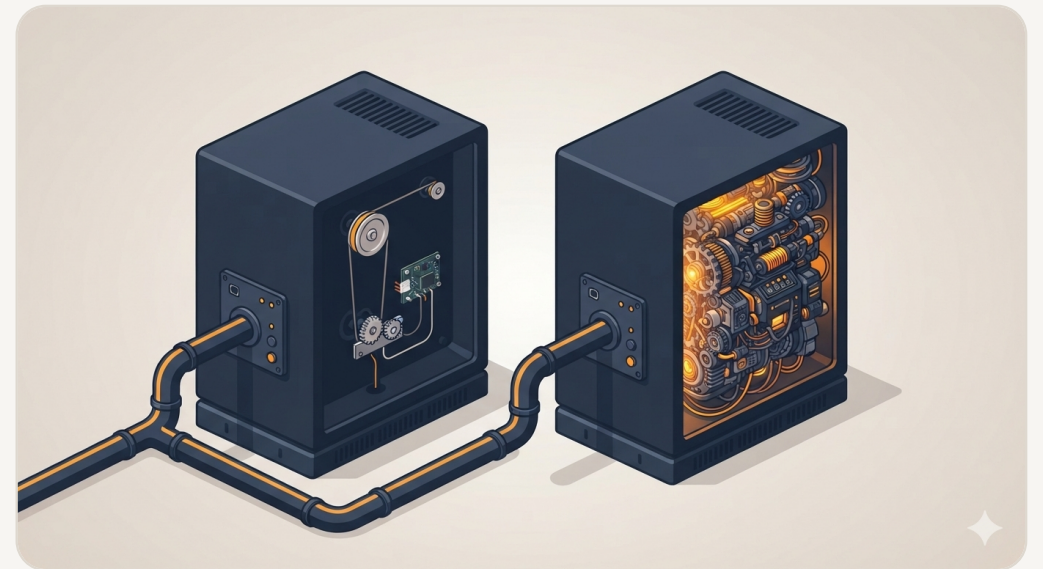
## PREREQUISITE

# The engine changed. The commands didn't.

---

```
# syntax=docker/dockerfile:1
```

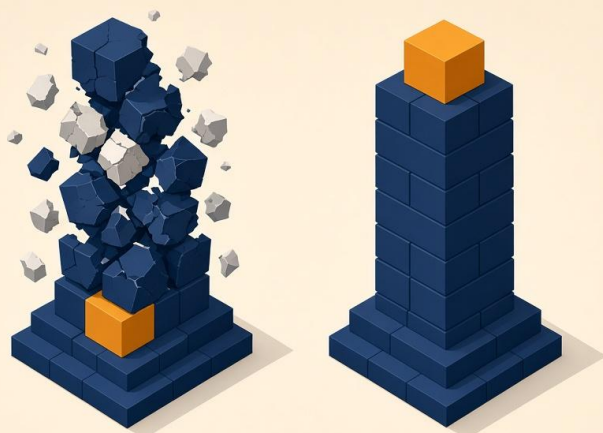
- Default builder since Docker 23
- This line pulls the **current** frontend, not the one bundled with your daemon
- Without it: no mounts, no heredoc, nothing below



## MOUNTS

# Cache without shipping the cache

---



```
RUN --mount=type=cache,target=/root/.cache/pip \
  pip install -r requirements.txt
```

- Survives between builds, never enters a layer
- Works for apt, npm, go, cargo, maven
- `sharing=locked` for tools that break on concurrent access

## MOUNTS

# A layer remembers everything

---

```
COPY token .  
RUN pip install --extra-index-url=...$(cat token)  
RUN rm token      ← too late
```

```
RUN --mount=type=secret,id=token \  
    pip install ...$(cat /run/secrets/token)
```

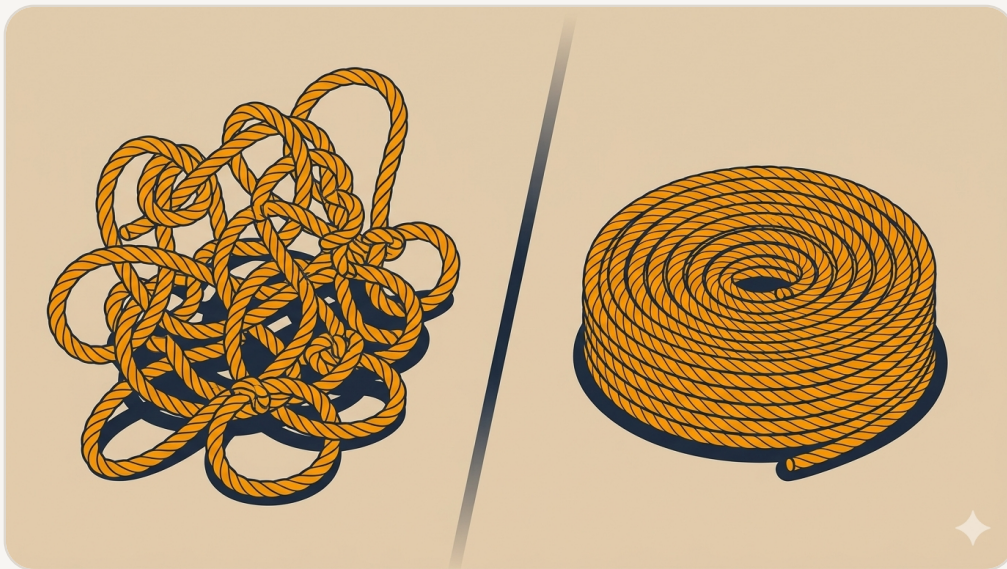
Deleting a file **hides** it. docker history still finds it.



## READABILITY

# A script is a script, not a chain

---



```
RUN apt-get update && apt-get install -y curl \  
    && rm -rf /var/lib/apt/lists/* \  
    && echo done
```

```
RUN <<EOF  
apt-get update  
apt-get install -y curl  
rm -rf /var/lib/apt/lists/*  
EOF
```

## TARGETS

# One Dockerfile, several images

---

```
FROM python:3.12-slim AS base
...
FROM base AS dev      # debugger, test tools
FROM base AS prod     # nothing extra
```

```
docker build --target dev -t app:dev .
docker build --target prod -t app:prod .
```

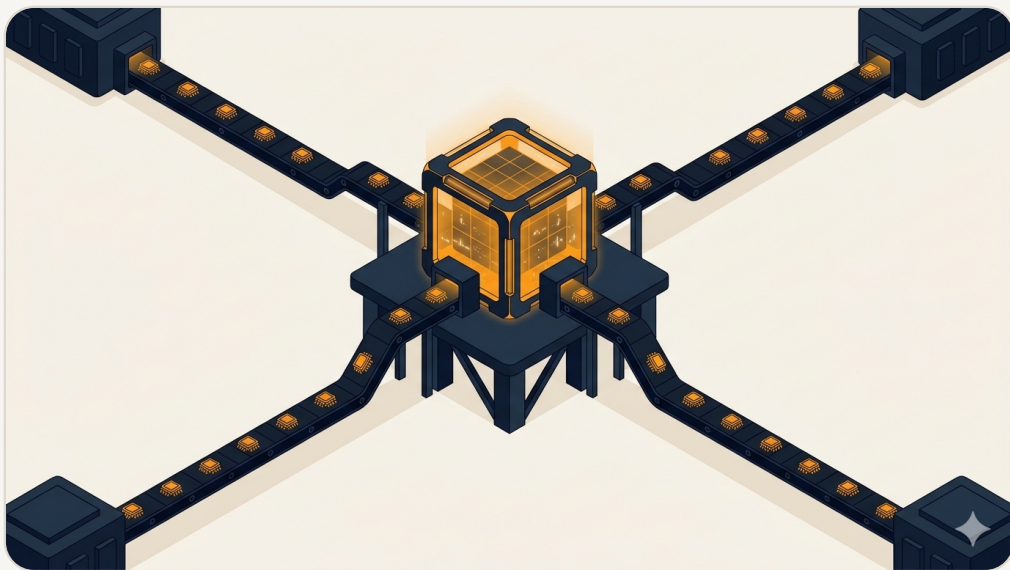
Two files drift apart within a month. One file cannot.



CI

## CI starts from nothing every time

---



```
docker build \
  --cache-from type=registry,ref=myrepo/app:cache \
  --cache-to   type=registry,ref=myrepo/app:cache,mode=max \
  -t myrepo/app:latest .
```

- Layer cache lives in the registry, not on the runner
- `mode=max` exports intermediate stages — without it the slow stage rebuilds anyway

## DEFAULTS

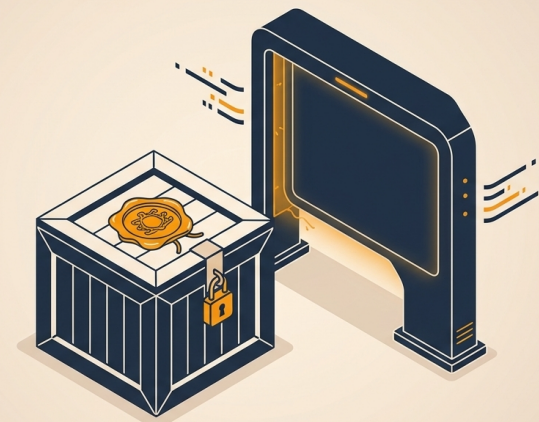
# buildx adds things you didn't ask for

- Provenance and SBOM manifests, attached by default

```
aws ecr describe-image-scan-findings  
ScanNotFoundException
```

```
docker build --provenance=false --sbom=false ...
```

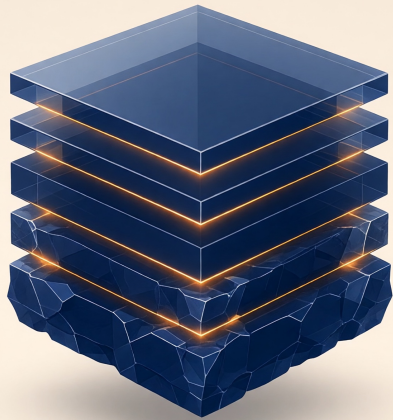
Scan-on-push was enabled. Nothing scanned, and **nothing warned**.



## INSPECTION

# Look inside an image you wrote yourself

---



```
efficiency: 98.1 %  
wasted:      6.3 MB
```

- `/var/lib/apt/lists/...`, `/var/log/apt/...`, debconf caches
- Created in one layer, overwritten in another, carried forever

## TAKEAWAYS

# Five things to try this week

---

- 1 `# syntax=docker/dockerfile:1` at the top of every Dockerfile
- 2 Cache mounts instead of choosing between fast and small
- 3 Secret mounts — a deleted file still ships
- 4 `--target` so dev and prod cannot drift apart
- 5 Registry cache in CI, with `mode=max`

